

Experiment No.1
Design & Implementation of Pass-1 of Two Pass Assembler
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology

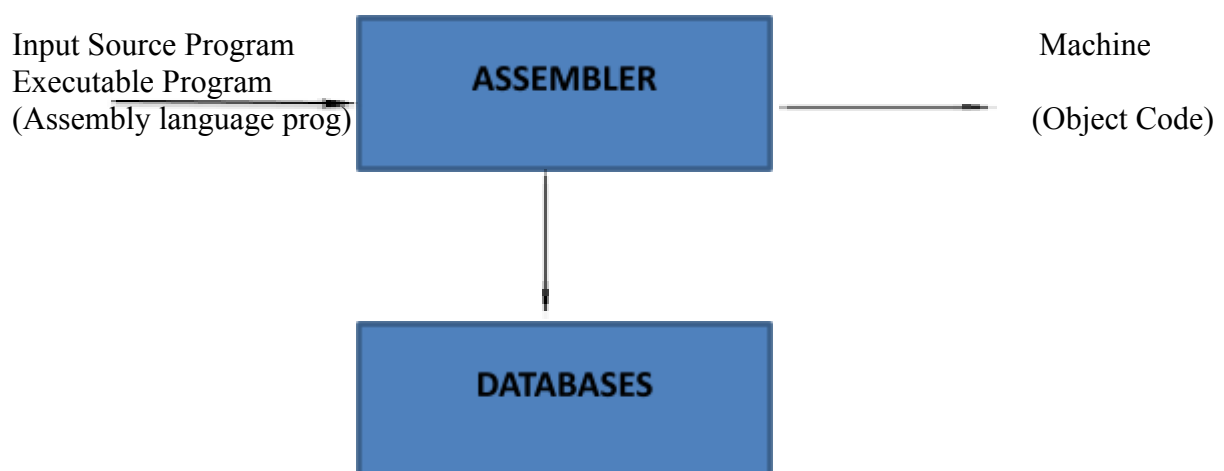
Department of Computer Engineering

Aim: Design & Implementation of Pass-1 of Two Pass Assembler.

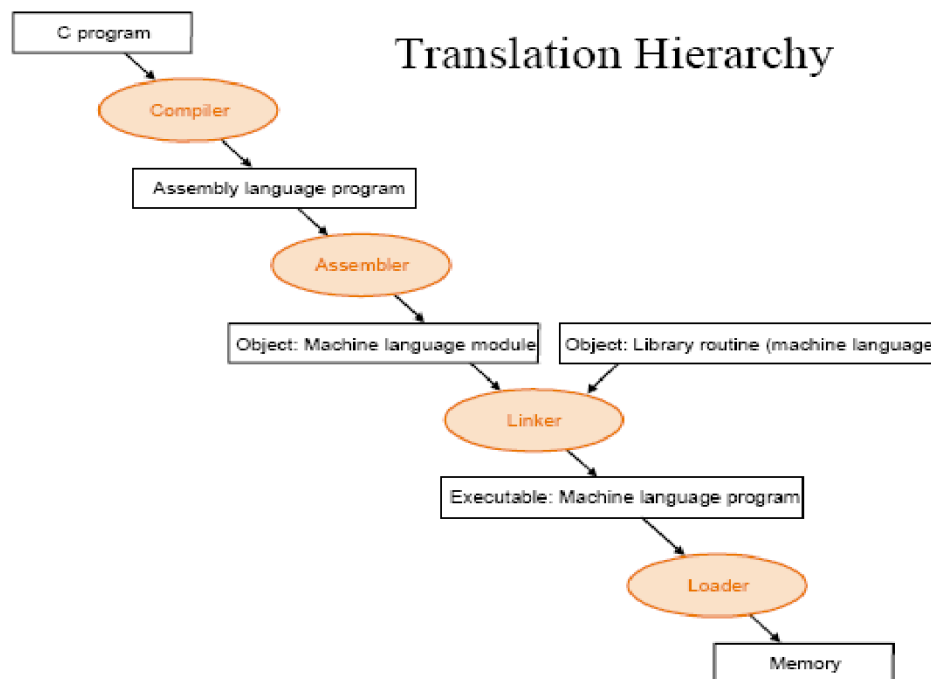
Objective: To study and implement Pass 1 of two pass assembler.

Theory:

An assembler is a program that accepts as input an assembly language program (source) and produces its machine language equivalent (object code) along with the information for the loader.



Relevance of Different system programs:



General Design Steps:

1. Specify the problem
2. Specify data structures
3. Define format of data structures
4. Specify algorithm
5. Look for modularity [capability of one program to be subdivided into independent programming units.]
6. Repeat 1 through 5 on modules

In following design IBM360 machine architecture is considered for reference.

Students should design 8086 / 8088 machine architecture.

1. Specify the problem

Pass1: Define symbols & literals.

- 1) Determine length of m/c instruction [MOTGET1]



- 2) Keep track of Location Counter [LC]
- 3) Remember values of symbols [STSTO]
- 4) Process some pseudo ops[EQU,DS etc] [POTGET1]
- 5) Remember Literals [LITSTO]

2. Data structure

Pass1: Databases

1. Input source program
2. "LC" location counter used to keep track of each instructions address.
3. M/c operation table (MOT) [Symbolic mnemonic & length]
4. Pseudo operation table [POT], [Symbolic mnemonic & action]
5. Symbol Table (ST) to store each lable & it's value.
6. Literal Table (LT), to store each literal (variable) & it's location.
7. Copy of input to used later by PASS-2.

3. Format of Data Structures

1. Machine Operation Table (MOT):

The op-code is the key and it's value is the binary op code equivalent, which is used for use in generating machine code.

The instruction length is stored for updating the location counter.

Instruction format is use in forming the m/c language equivalent.



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

← 6 bytes entry →				
Mnemonic Op-code (4 byte) character	Binary Op-code (1 byte) Hex	Instruction length (2 bits) (binary)	Instruction format (3 bits) (binary)	Not used in this design (3 bits)
"Abbb"	5A	10	001	-
"LOAD"	4A	10	001	-
"MOVb"	5E	01	000	-
"MVIb"	1E	11	100	-

b → represents blank

Codes used-

Instruction length

01 = 1 half words

10 = 2 half words

11 = 3 half words

Instruction

000 - RR

001 - RX

010 - RS

011 - SI

100 - SS

2. Pseudo Operation Table (POT):

← 8 bytes / entry →	
Pseudo-op (5-bytes) (character)	Address of routine to process pseudo-op 3 bytes = 24 bit
"EQUbb"	P1EQG
"ENDbb"	P1END
"START"	P1START
"USING"	P1USING

← These are presumably
labels of routines in pass1

3. Symbol table & Literal table:-



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

14 bytes / entry			
Symbol 8 byte character	Value (4 byte) Hex	Length 1 byte Hex	Relocation 1 byte character
"LOOPbbbb"	0000	01	"R"
"JOHNbbbb"	0010	04	"R"

Relation – R- Relative

A- Absolute (i.e. does not change)

4. Base table (BT):

4 bytes / entry			
	Availability Indicator 1 byte Character	Designated relative address contents of base register (3 bytes = 24bit address) Hex	-
1	"N"	-	↑ 15 entries ↓
2	"N"	-	
3	"N"	-	
.	:	-	
.	:	-	
15	"Y"	00 00 00	

Codes:-

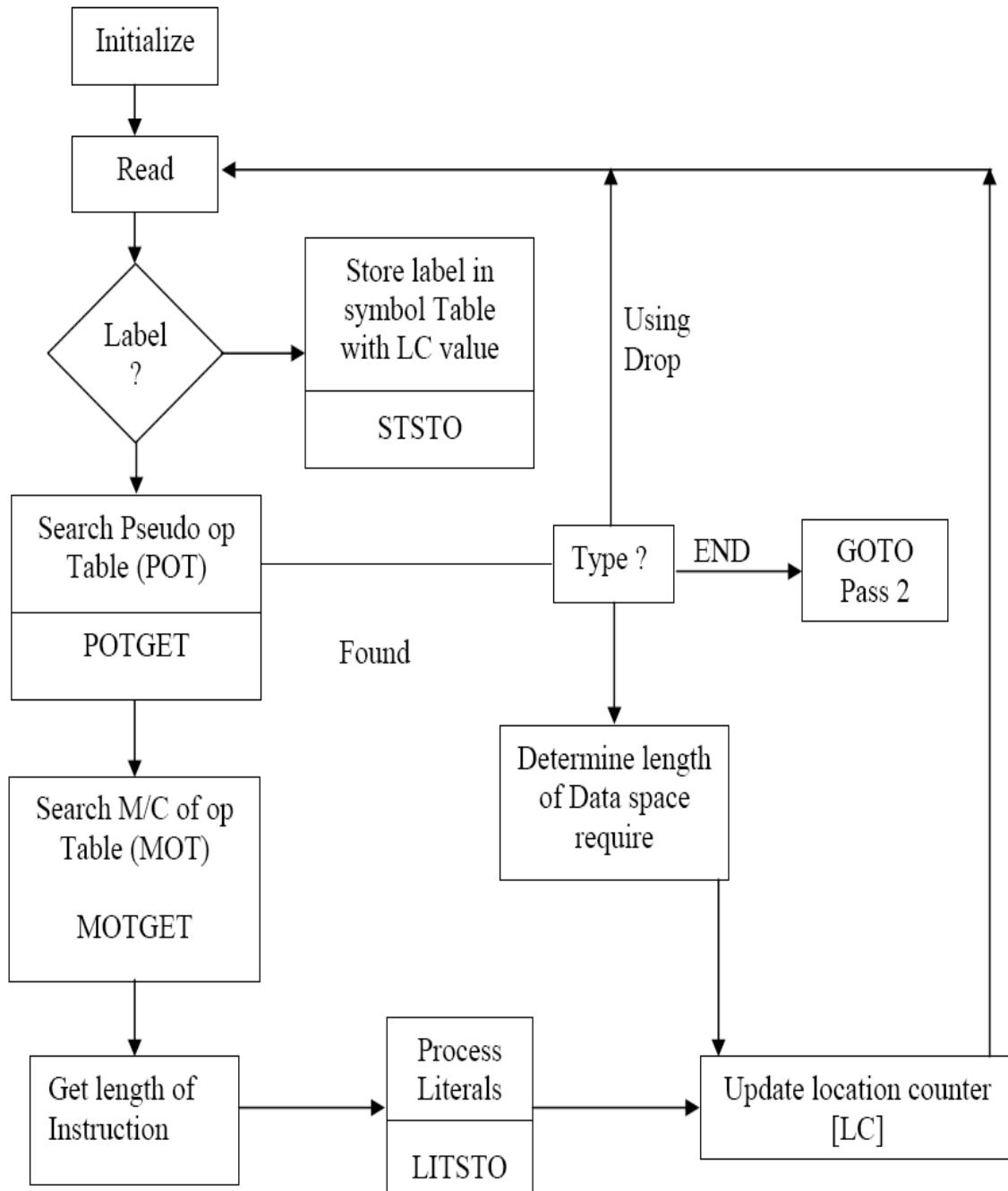
Availability:-

Y = register specified in USING pseudo-op

N = register never specified in USING pseudo-op or subsequently made unavailable by Drop pseudo-op.



Flow Chart of Pass 1 of Two Pass Assembler





Code:

main.c

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
void main() {
    char opcode[10], operand[10], label[10], code[10], mnemonic[10];
    int locctr, start, length;

    FILE *fp1, *fp2, *fp3, *fp4;

    fp1 = fopen("input.txt", "r");
    fp2 = fopen("optab.txt", "r");
    fp3 = fopen("symtbl.txt", "w");
    fp4 = fopen("out.txt", "w");

    fscanf(fp1, "%s\t%s\t%s", label, opcode, operand);

    if(strcmp(opcode, "START")==0) {
        start = atoi(operand);
        locctr = start;
        fprintf(fp4, "\t%s\t%s\t%s\n", label, opcode, operand);
        fscanf(fp1, "%s\t%s\t%s", label, opcode, operand);
    }
    else {
```



```
locctr = 0;
}

while(strcmp(opcode, "END")!=0) {
    fprintf(fp4, "%d\t", locctr);

    if(strcmp(label, "**")!=0) {
        fprintf(fp3, "%s\t%d\n", label, locctr);
    }

    fscanf(fp2, "%s\t%s", code, mnemonic);

    while(strcmp(code, "END")!=0) {
        if(strcmp(opcode, code)==0) {
            locctr+=3;
            break;
        }
        fscanf(fp2, "%s\t%s", code, mnemonic);
    }

    if(strcmp(opcode, "WORD")==0) {
        locctr+=3;
    }
    else if(strcmp(opcode, "RESW")==0) {
        locctr+=(3*(atoi(operand)));
    }
}
```



```
else if(strcmp(opcode, "RESB")==0) {
    locctr+=(atoi(operand));
}
else if(strcmp(opcode, "BYTE")==0) {
    ++locctr;
}

fprintf(fp4, "%s\t%s\t%s\t\n", label, opcode, operand);
fscanf(fp1, "%s\t%s\t%s", label, opcode, operand);
}

fprintf(fp4, "%d\t%s\t%s\t%s\n", locctr, label, opcode, operand);

length = locctr-start;

printf("The length of the code : %d\n", length);

fclose(fp1);
fclose(fp2);
fclose(fp3);
fclose(fp4);
}
```

input.txt

```
** START 2000
** LDA FIVE
```



```
** STA ALPHA
** LDCH  CHARZ
** STCH  C1
ALPHA RESW  2
FIVE  WORD  5
CHARZ BYTE  C'Z'
C1 RESB  1
** END **
```

optab.txt

```
START *
LDA 03
STA 0f
LDCH 53
STCH 57
END *
```

Output:

symtbl.txt

```
ALPHA      2012
FIVE  2018
CHARZ      2021
C1         2022
```



out.txt

```

      **      START      2000
2000  **      LDA   FIVE
2003  **      STA   ALPHA
2006  **      LDCH  CHARZ
2009  **      STCH  C1
2012  ALPHA      RESW 2
2018  FIVE  WORD      5
2021  CHARZ      BYTE C'Z'
2022  C1      RESB 1
2023  **      END      **
```

Conclusion: Pass 1 of the assembler lays the groundwork for Pass 2 by generating essential databases like the symbol table and calculating memory addresses for each instruction. These databases are crucial for generating the final machine code during Pass 2, where symbolic addresses are resolved and translated into actual memory addresses.

Experiment No. 2
Design & Implementation of Pass-2 of Two Pass Assembler
Date of Performance:
Date of Submission:



Aim: Design and Implementation of Pass 2 of Two Pass Assembler.

Objective: To study and implement Pass 2 of two pass assembler for IBM 360 Machine.

Theory:

An assembler is a program that accepts as input an assembly language program (source) and produces its machine language equivalent (object code) along with the information for the loader. The function of pass 2 of assembler is to generate machine program which is semantically equivalent to source program by taking input from pass 1. Databases generated are used to generate object code in pass 2.

Pass2: Generate object program

1. Look up value of symbols [STGET]
2. Generate instruction [MOTGET2]
3. Generate data (for DS, DC & literals)
4. Process pseudo ops [POTGET2]

Pass2: Databases

1. Copy of source program input to Pass1.
2. Location Counter (LC)
3. MOT [Mnemonic, length, binary m/c op code, etc.]
4. POT [Mnemonic & action to be taken in Pass2]
5. ST [prepared by Pass1, label & value]
6. Base Table [or register table] indicates which registers are currently specified using 'USING' pseudo op & what are contents.
7. Literal table prepared by Pass1. [Lit name & value].



Flowchart of Pass 2:

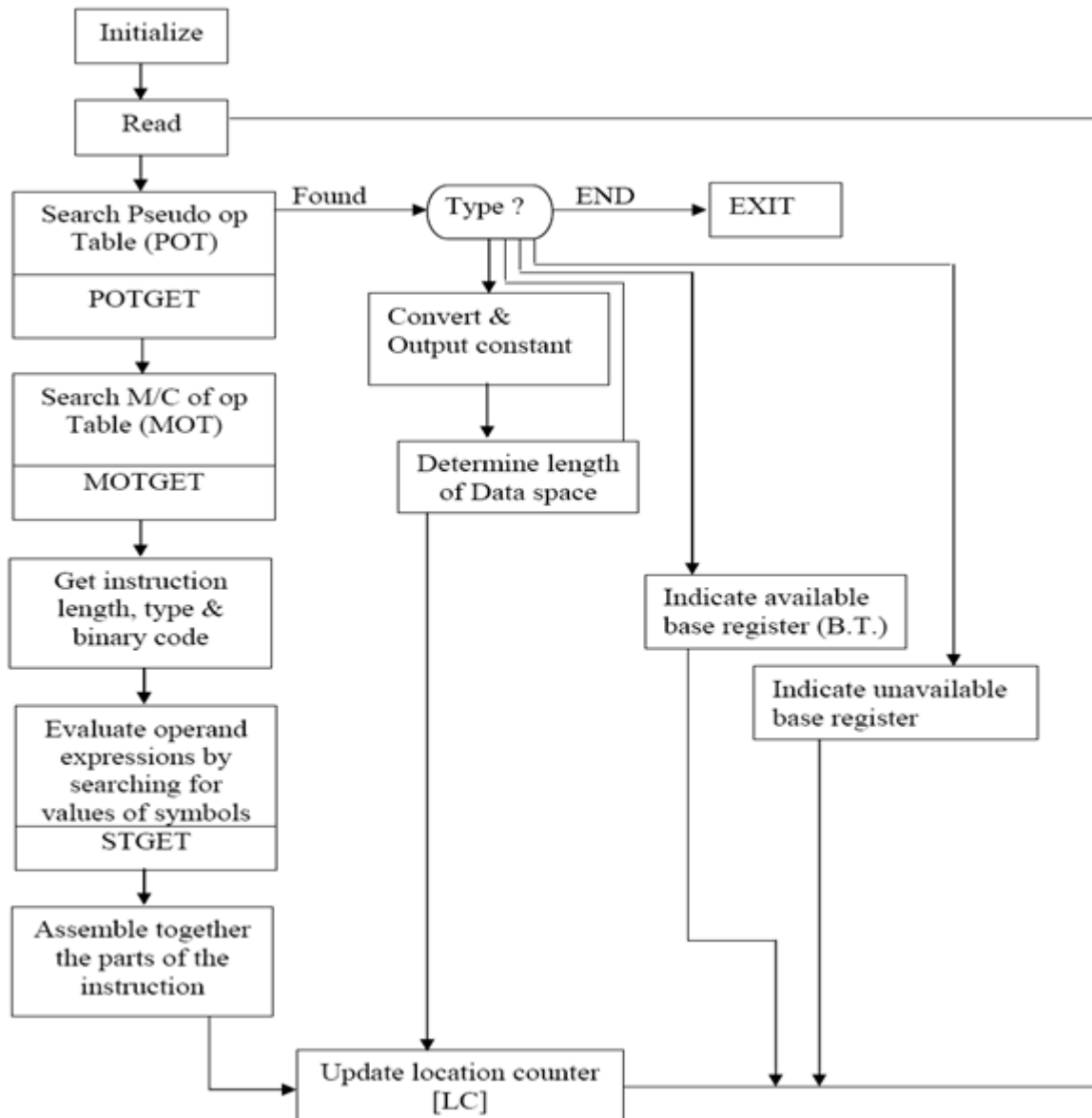


Fig. Flow chart of Pass -2

Code:

main.c

```
#include <stdio.h>
```

```
#include <string.h>
```



```
int findAddress(int op) {
    FILE *INTERMEDIATE;
    char opcode[22], operand[33], label[43], address[10];
    int finalAddress = 0, count = 0, start;
    INTERMEDIATE = fopen("output.dat", "r");
    if (INTERMEDIATE == NULL) {
        printf("Error opening output.dat file.\n");
        return -1;
    }
    fscanf(INTERMEDIATE, "%s%s%s", label, opcode, operand);
    start = atoi(operand);
    while (strcmp(opcode, "END") != 0) {
        fscanf(INTERMEDIATE, "%s%s%s%s", address, label, opcode, operand);
        if (strcmp(opcode, "RESW") == 0)
            count += 3 * atoi(operand);
        if (strcmp(opcode, "RESB") == 0)
            count += atoi(operand);
        finalAddress = atoi(address);
    }
    fclose(INTERMEDIATE);
    if (op == 0)
        return finalAddress;
    return finalAddress - start - count;
}

int findInSYMTAB(char findLabel[]) {
```



```
FILE *SYMTAB;

char label[30], addr[30];

SYMTAB = fopen("symtab.dat", "r");

if (SYMTAB == NULL) {
    printf("Error opening symtab.dat file.\n");
    return -1;
}

fscanf(SYMTAB, "%s%s", label, addr);

while (!feof(SYMTAB)) {
    if (strcmp(findLabel, label) == 0) {
        fclose(SYMTAB);
        return atoi(addr);
    }

    fscanf(SYMTAB, "%s%s", label, addr);
}

fclose(SYMTAB);

return -1;
}
```

```
int getMnemonicCode(char mnemonic[]) {
    if (strcmp(mnemonic, "LDA") == 0)
        return 33;
    else if (strcmp(mnemonic, "STA") == 0)
        return 44;
    else if (strcmp(mnemonic, "LDCH") == 0)
        return 53;
}
```



```
else if (strcmp(mnemonic, "STCH") == 0)
    return 57;
else
    return -1;
}

void main() {
    FILE *INTERMEDIATE, *SYMTAB, *FINAL, *OBJ;
    int finalAddress = 0, length, i, startAddr;
    char opcode[22], operand[33], label[43], address[10];

    SYMTAB = fopen("symtab.dat", "r");
    if (SYMTAB == NULL) {
        printf("Error opening symtab.dat file.\n");
        return;
    }
    FINAL = fopen("final.dat", "w");
    if (FINAL == NULL) {
        printf("Error opening final.dat file.\n");
        fclose(SYMTAB);
        return;
    }
    OBJ = fopen("obj.dat", "w");
    if (OBJ == NULL) {
        printf("Error opening obj.dat file.\n");
        fclose(SYMTAB);
```



```
fclose(FINAL);

return;
}

finalAddress = findAddress(0);

INTERMEDIATE = fopen("output.dat", "r");
if (INTERMEDIATE == NULL) {
    printf("Error opening output.dat file.\n");
    fclose(SYMTAB);
    fclose(FINAL);
    fclose(OBJ);
    return;
}

fscanf(INTERMEDIATE, "%s%s%s", label, opcode, operand);

startAddr = atoi(operand);
if (strcmp(opcode, "START") == 0) {
    fprintf(FINAL, "%s\t%s\t%s\n", label, opcode, operand);
    length = finalAddress - startAddr;
    fprintf(OBJ, "H^%s^00%s^00%d\n", label, operand, length);
    fscanf(INTERMEDIATE, "%s%s%s%s", address, label, opcode, operand);
    fprintf(OBJ, "T^%06d^%d^", atoi(address), findAddress(1));
}
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

```
while (strcmp(opcode, "END") != 0) {
    if (strcmp(label, "**") == 0) {
        fprintf(OBJ, "%d%d^", getMnemonicCode(opcode), findInSYMTAB(operand));
        fprintf(FINAL, "%s\t%s\t%s\t%s\t%d%\n", address, label, opcode, operand,
getMnemonicCode(opcode), findInSYMTAB(operand));
    } else if (strcmp(opcode, "BYTE") == 0) {
        fprintf(FINAL, "%s\t%s\t%s\t%s", address, label, opcode, operand);
        for (i = 2; i < strlen(operand) - 1; i++) {
            fprintf(OBJ, "%x", operand[i]);
            fprintf(FINAL, "%x", operand[i]);
        }
        fprintf(FINAL, "\n");
    } else if (strcmp(opcode, "WORD") == 0) {
        fprintf(OBJ, "%06x^", atoi(operand));
        fprintf(FINAL, "%s\t%s\t%s\t%s\t%06x\n", address, label, opcode, operand,
atoi(operand));
    }
    fscanf(INTERMEDIATE, "%s%s%s%s", address, label, opcode, operand);
}

fprintf(FINAL, "%s\t%s\t%s\t%s\n", address, label, opcode, operand);
fprintf(OBJ, "\nE^%06d\n", startAddr);

fclose(INTERMEDIATE);
fclose(SYMTAB);
fclose(FINAL);
fclose(OBJ);
```



}

output.dat

```
      **      START      2000
2000  **      LDA   FIVE
2003  **      STA   ALPHA
2006  **      LDCH  CHARZ
2009  **      STCH  C1
2012  ALPHA      RESW 2
2018  FIVE  WORD      5
2021  CHARZ      BYTE C'Z'
2022  C1      RESB 1
2023  **      END    **
```

symtab.dat

```
ALPHA      2012
FIVE  2018
CHARZ      2021
C1      2022
```

Output:

obj.dat

```
H^**^002000^0023
T^002000^16^332018^442012^532021^572022^000005^5a
E^002000
```



final.dat

```
**      START      2000
2000  **      LDA   FIVE 332018
2003  **      STA   ALPHA      442012
2006  **      LDCH  CHARZ      532021
2009  **      STCH  C1      572022
2018  FIVE  WORD      5      000005
2021  CHARZ      BYTE C'Z'5a
2023  **      END   **
```

Conclusion:

In conclusion, Pass 2 of a two-pass assembler translates assembly language instructions into machine code or generates object code. It utilizes databases generated during Pass 1, such as the symbol table, to resolve labels and addresses. The program reads the intermediate file generated by Pass 1, processes each instruction, generates machine or object code, and produces a final listing file and an object code file. Pass 2 ensures the accuracy and correctness of the assembled program, making it executable on the target machine.

Experiment No.3
Design & Implementation of pass 1 of Two Pass Macro Processor
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: Design & Implementation of Pass 1 of Two Pass Macro Processor.

Objective: To study and implement Pass 1 of two pass Macro Processor for IBM 360 Machine.

Theory:

Macro: A macro is a unit of specification for program generation through expansion. A macro instruction is a notational convenience for the programmer. It allows the programmer to write shorthand version of a program (module programming).

Macro Processor: The macro processor replaces each macro invocation with the corresponding sequence of statements.

You may design a two-pass macro processor.

Pass 1: Process all macro definitions.

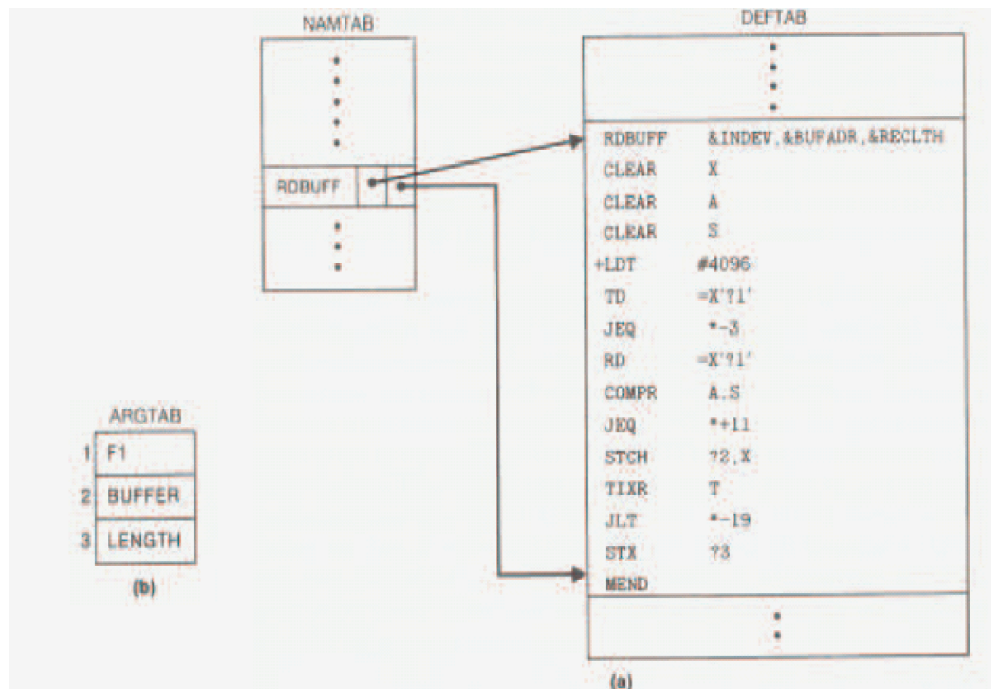
- Identifies macro definitions and calls in the program
- Determine the formal parameters & their values

Pass 1 Databases of Macro Processor

1. Copy of source program statements
2. Output macro source listings for use by Pass-2
3. MDT to store body of macro definition
4. MDTC used to points the next entry towards in MDT
5. NMT to store names of macro defines in the program
6. MNTC used to points towards next entry into MNT.
7. An array called Parameter List Array (PLA) to manage an index for formal parameters (arguments)



Data structure



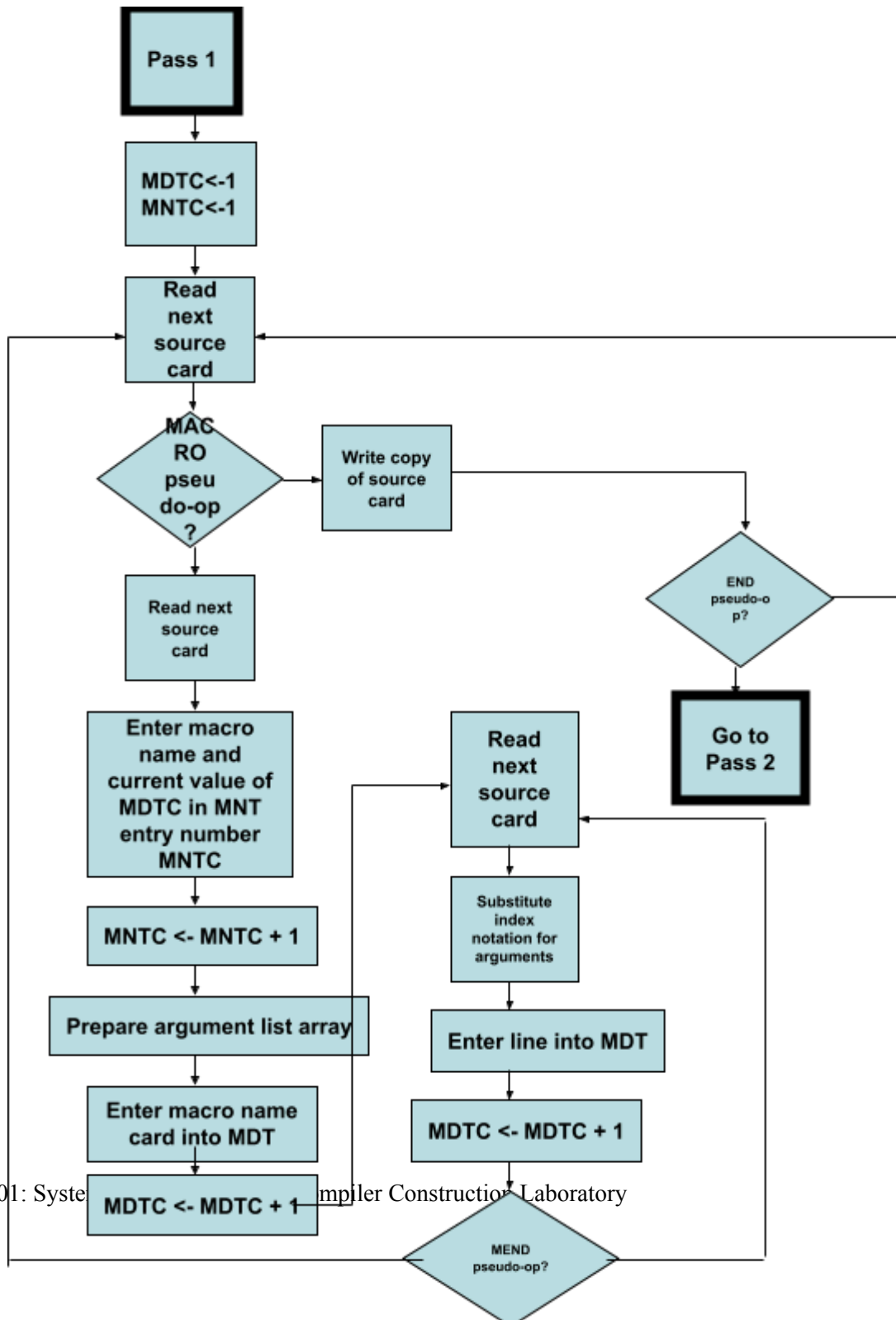
PASS-I of Macro Processor- Processing Macro Definitions

1. Initialize MDTC and MNTC
2. Read the next source statement of the program.
3. If the statement contains MACRO pseudo-op. go to step 6.
4. Output the instruction of the statement.
5. If the statement contains END pseudo-op, go to PASS-II else go to step 2
6. Read the next source statement of the program.
7. Make an entry of the macro name and MTDC in MNT at location MNTC and increment MNTC by 1.
8. Prepare the parameter (arguments) list array.
9. Enter macro name into MDT and increment MTDC by 1.
10. Read the next card and substitute index for the parameters (arguments).
11. Enter line into MDT and increment MDT by 1.



12. If MEND pseudo-op is found, go to step 2 else go to step 10

Pass 1 flowchart:





Code:

main.java

```
import java.util.*;
import java.io.*;
class MACRO
{
static String mnt[][]=new String[5][3]; //assuming 5 macros in 1

static String ala[][]=new String[10][2]; //assuming 2 arguments in

static String mdt[][]=new String[20][1]; //assuming 4 LOC for each

static int mntc=0,mdtc=0,alac=0;
public static void main(String args[])
{
pass1();
System.out.println("\n*****PASS-1 MACROPROCESSOR*****\n");
System.out.println("MACRO NAME TABLE (MNT)\n");
System.out.println("i macro loc\n");
display(mnt,mntc,3);
System.out.println("\n");
System.out.println("ARGUMENT LIST ARRAY(ALA) for Pass1\n");
display(ala,alac,2);
System.out.println("\n");
System.out.println("MACRO DEFINITION TABLE (MDT)\n");
```



```
display(mdt,mdtc,1);
System.out.println("\n");
}
static void pass1()
{
int index=0,i;
String s,prev="",substring;
try
{
BufferedReader inp = new BufferedReader(new FileReader("input.txt"));
File op = new File("pass1_output.txt");
if (!op.exists())
op.createNewFile();
BufferedWriter output = new BufferedWriter(new
FileWriter(op.getAbsolutePath()));
while((s=inp.readLine())!=null)
{
if(s.equalsIgnoreCase("MACRO"))
{
prev=s;
for(;!(s=inp.readLine()).equalsIgnoreCase("MEND");mdtc++,prev=s)
{
if(prev.equalsIgnoreCase("MACRO"))
{
StringTokenizer st=new StringTokenizer(s);
String str[]=new String[st.countTokens()];
```



```
for(i=0;i<str.length;i++)
str[i]=st.nextToken();
mnt[mntc][0]=(mntc+1)+""; //mnt formation
mnt[mntc][1]=str[0];
mnt[mntc++][2]=(++mdtc)+"";
st=new StringTokenizer(str[1],","); //tokenizing the arguments
String string[]=new String[st.countTokens()];
for(i=0;i<string.length;i++)
{
string[i]=st.nextToken();
ala[alac][0]=alac+""; //ala table formation
index=string[i].indexOf("=");
if(index!=-1)
ala[alac++][1]=string[i].substring(0,index);
else
ala[alac++][1]=string[i];
}
}
else //automatically eliminates tagging of arguments in definition
{ //mdt formation
index=s.indexOf("&");
substring=s.substring(index);
for(i=0;i<alac;i++)
if(ala[i][1].equals(substring))
s=s.replaceAll(substring,"#+ala[i][0]);
}
```



```
mdt[mdtc-1][0]=s;
}
mdt[mdtc-1][0]=s;
}
else
{
output.write(s);
output.newLine();
}
}
output.close();
}
catch(FileNotFoundException ex)
{
System.out.println("UNABLE TO END FILE ");
}
catch(IOException e)
{
e.printStackTrace();
}
}
static void display(String a[],int n,int m)
{
int i,j;
for(i=0;i<n;i++)
{
```



```
for(j=0;j<m;j++)  
System.out.print(a[i][j]+" ");  
System.out.println();  
}  
}  
}
```

input.txt

```
START  
MACRO  
INCR &ARG3 &ARG2  
ADD AREG &ARG1  
MOVER BREG &ARG1  
MEND  
MACRO  
PVG &ARG2 &ARG1  
SUB AREG &ARG2  
MOVER CREG & ARG1  
MEND  
INCR  
DECR  
DATA2  
END
```



Output:

*****PASS-1 MACRO PROCESSOR*****

MACRO NAME TABLE (MNT)

i macro loc

1 INCR 1

2 PVG 5

ARGUMENT LIST ARRAY(ALA) for Pass1

0 &ARG3

1 &ARG2

MACRO DEFINITION TABLE (MDT)

INCR &ARG3 &ARG2

ADD AREG &ARG1

MOVER BREG &ARG1

MEND

PVG &ARG2 &ARG1

SUB AREG #1

MOVER CREG & ARG1

MEND

pass1_output.txt

START

INCR

DECR

DATA2

END



Conclusion:

Pass 1 of a macro processor scans source code, collects macro definitions, handles parameters, and builds a database storing macro names, parameters, replacement text, and other metadata. This database aids in efficient macro expansion during subsequent passes.

Experiment No.4
Design & Implementation of Pass 2 of Two Pass Macro Processor
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: Design & Implementation of Pass 2 of Two Pass Macro Processor.

Objective: To study and implement Pass 2 of two pass Macro Processor for IBM 360 Machine.

Theory:

Macro: A macro is a unit of specification for program generation through expansion. A macro instruction is a notational convenience for the programmer. It allows the programmer to write shorthand version of a program (module programming).

Macro Processor: The macro processor replaces each macro invocation with the corresponding sequence of statements.

Pass 2: Expand all macro invocation statements.

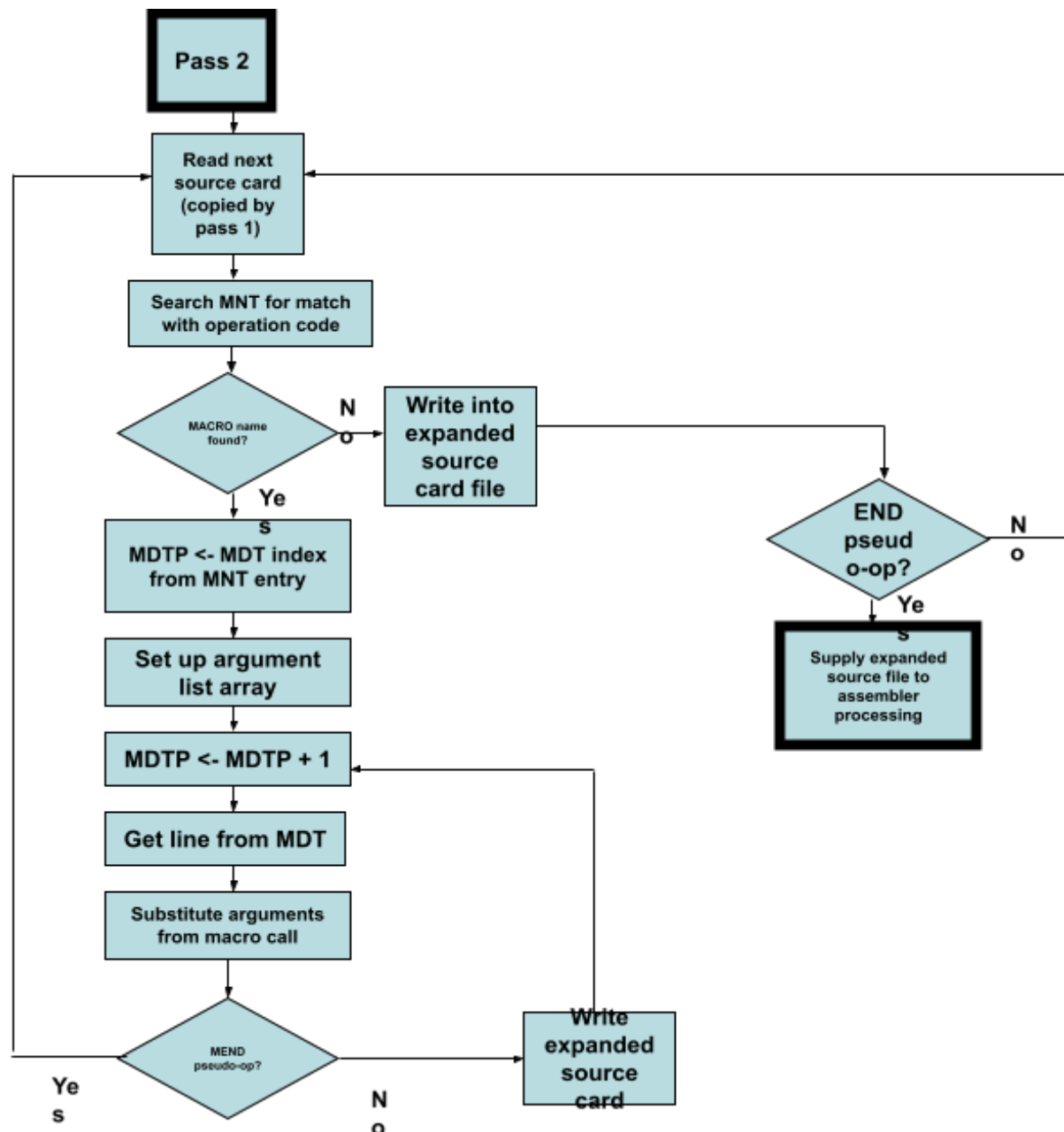
- Monitors the values of expansion time variables & sequencing symbols specified in a Macro
- Handles expansion time control flow & performs expansion of model statements.

Pass 2 Databases of Macro Processor

1. Copy of source program statements
2. Output expanded source listings for input to assembler
3. MNT and MDT generated in pass 1
4. Macro Definition Table Pointer (MDTP) directs to the next statement to be used during expansion
5. An array called Parameter List Array (PLA) for substituting parameters of macro call in the source macro of the macro call in the stored macro definitions for the index correspondingly.



Pass 2 flowchart:





Code:

main.c

```
#include <stdio.h>
#include <string.h>

int main() {
    FILE *f1, *f2, *f3, *f4, *f5;
    int i, len;
    char mne[20], opnd[20], la[20], name[20], mne1[20], opnd1[20], arg[20];

    f1 = fopen("minp2.txt", "r");
    f2 = fopen("ntab2.txt", "r");
    f3 = fopen("dtab2.txt", "r");
    f4 = fopen("atab2.txt", "w+");
    f5 = fopen("op2.txt", "w");

    while (fscanf(f1, "%s%s%s", la, mne, opnd) != EOF) {
        if (strcmp(mne, "END") == 0) {
            break;
        }

        if (strcmp(mne, "MACRO") == 0) {
            while (fscanf(f1, "%s%s%s", la, mne, opnd) != EOF) {
                if (strcmp(mne, "MEND") == 0)
                    break;
            }
        }
    }
}
```



```
} else {  
    fscanf(f2, "%s", name);  
    if (strcmp(mne, name) == 0) {  
        len = strlen(opnd);  
        for (i = 0; i < len; i++) {  
            if (opnd[i] != ',')  
                fprintf(f4, "%c", opnd[i]);  
            else  
                fprintf(f4, "\n");  
        }  
        fseek(f2, 0, SEEK_SET);  
        fseek(f4, 0, SEEK_SET);  
        fscanf(f3, "%s%s", mne1, opnd1);  
        fprintf(f5, ".\t%s\t%s\n", mne1, opnd);  
        fscanf(f3, "%s%s", mne1, opnd1);  
        while (strcmp(mne1, "MEND") != 0) {  
            if ((opnd1[0] == '&')) {  
                fscanf(f4, "%s", arg);  
                fprintf(f5, "-\t%s\t%s\n", mne1, arg);  
            } else  
                fprintf(f5, "-\t%s\t%s\n", mne1, opnd1);  
            fscanf(f3, "%s%s", mne1, opnd1);  
        }  
    } else  
        fprintf(f5, "%s\t%s\t%s\n", la, mne, opnd);  
}
```



```
}  
fprintf(f5, "%s\t%s\t%s\n", la, mne, opnd);  
fclose(f1);  
fclose(f2);  
fclose(f3);  
fclose(f4);  
fclose(f5);  
  
printf("Pass2\n");  
return 0;  
}
```

minp2.txt

```
EX1  MACRO      &A,&B  
-    LDA  &A  
-    STA  &B  
-    MEND-  
  
SAMPLE  START      1000  
-    EX1  N1,N2  
  
N1     RESW 1  
N2     RESW 1  
-    END  -
```

dtab2.txt

```
EX1  &A,&B  
  
LDA  &A
```



STA &B

MEND

ntab2.txt

EX1

Output:

atab2.txt

N1

N2

op.txt

SAMPLE START 1000

. EX1 N1,N2

- LDA N1

- STA N2

N1 RESW 1

N2 RESW 1

- END -

Conclusion:

The program performs Pass 2 of the macro processing, expanding macro invocations and generating the final output with expanded instructions. It handles macro definitions, macro invocations, and substitution of formal parameters effectively, providing a basis for further processing or compilation of the source code.